

An
Industry Oriented Mini Project Report
on
Autism spectrum disorder analysis for toddlers

Submitted in partial fulfillment of the requirements for the award of degree

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

Submitted By

SK. IRFAN BABA	227Z1A66B5
V.SIRI VENNALA	227Z1A66C6
S.DARSHAN	227Z1A66B4

Under the Guidance of

Mrs. G. MANASA
Assistant Professor



SCHOOL OF ENGINEERING

Department of Computer Science and Engineering
(Artificial intelligence and machine learning)

NALLA NARASIMHA REDDY
EDUCATION SOCIETY'S GROUP OF INSTITUTIONS
(AN AUTONOMOUS INSTITUTION)

Approved by AICTE, New Delhi, Chowdariguda (V) Korremula 'x'Roads,
via Narapally, Ghatkesar (Mandal) Medchal (Dist), Telangana-500088

2024-2025

SCHOOL OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report titled “**AUTISM SPECTRUM DISORDER ANALYSIS FOR TODDLERS**” is being submitted by **Sk Irfan Baba (227Z1A66B5), V. Siri Vennela (227Z1A66C6) and S.Darshan (227Z1A66B4)** in Partial fulfillment for the award of **Bachelor of technology in Computer Science & Engineering** is a record bonafide work carried out by them. The results embodied in this report have not been submitted to any other University for the award of any degree.

Internal Guide

(Mrs.G.Manasa)

Head of the Department

(Dr.G.Sravan kumar)

Submitted for the University Examination held on.....

External Examiner

DECLARATION

We SK.Irfan , V.Siri Vennela and S.Darshan the students of **Bachelor of Technology in Computer Science and Engineering, Nalla Narasimha Reddy Education Society's Group of Institutions**, Hyderabad, Telangana, hereby declare that the work presented in this project work entitled **AUTISM SPECTRUM DISORDER ANALYSIS FOR TODDLERS** is the outcome of our own bonafide work and is correct to the best of our knowledge and this work has been undertaken taking care of engineering ethics. It contains no material previously published or written by another person nor material which has been accepted for the award of any other degree or diploma of the university or other institute of higher learning.

SK. IRFAN BABA	227Z1A66B5
V.SIRI VENNALA	227Z1A66C6
S.DARSHAN	227Z1A66B4

Date:

Signature:

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide **Mrs. G. Manasa**, Assistant Professor, Department of Computer Science and Engineering, NNRESGI, who motivated throughout the period of the project and also for his valuable and intellectual suggestions, guidance, and constant encouragement right throughout our work.

We would like to express our profound gratitude to our project coordinator **Mr. SAMUEL CHEPURI**, Assistant Professor, Department of Computer Science and Engineering, NNRESGI, for his support and guidance in completing our project and for giving us this opportunity to present the project work.

We profoundly express thanks to **Dr. G .SRAVAN KUMAR**, Professor & Head, Department of Computer Science and Engineering, NNRESGI, for his cooperation and encouragement in completing the project successfully.

We wish to express our sincere thanks to **Dr. G. Janardhana Raju**, Dean School of Engineering, NNRESGI, for providing the facilities for completion of the project.

We wish to express our sincere thanks to **Dr. C. V. Krishna Reddy**, Director, NNRESGI, for providing the facilities for completion of the project.

Finally, we would like to thank Project Review Committee (PRC) members, all the faculty members and supporting staff, Department of Computer Science and Engineering, NNRESGI, for extending their help in all circumstances.

By

SK. IRFAN BABA 227Z1A66B5

V.SIRI VENNALA 227Z1A66C6

S.DARSHAN 227Z1A66B4

ABSTRACT

Autism Spectrum Disorder (ASD) is a neuro-developmental condition that affects communication, social interaction, and behavior. Early detection plays a crucial role in improving outcomes through timely interventions. This project aims to develop a web-based autism screening tool that collects responses to 10 key behavioral questions and uses a trained machine learning (ML) model to assess ASD risk levels. Based on the responses, the system will classify the result as "Mild," "Severe," or "No ASD Traits" and provide precautionary recommendations. The project integrates data preprocessing, feature selection, and predictive modeling to enhance accuracy. A user-friendly interface will allow parents, caregivers, and clinicians to conduct preliminary assessments. The system leverages historical screening data to train an ML model, ensuring reliable predictions. The model will be optimized for accuracy and efficiency, enabling real-time responses on the web interface. Additionally, it will provide users with precautionary steps based on the severity of detected ASD traits, guiding them toward appropriate next steps, such as seeking professional evaluation. By combining technology with healthcare screening, this project aims to offer an accessible and effective early intervention tool for autism awareness and detection.

keywords: Autism Screening, Machine Learning, ASD Detection, Early Diagnosis, Behavioral Analysis, Expected Treatment Level.

TABLE OF CONTENTS

	Page No.
Abstract	
List of Figures	i
List of Tables	ii
List of Abbreviations	iii
1. INTRODUCTION	1
1.1 Motivation	1
1.2 Problem Definition	2
1.3 Objective of project	2
1.4 Limitation of project	2
2. LITERATURE SURVEY	4
2.1 Introduction	4
2.2 Existing System	5
2.3 Proposed System	6
3. SYSTEM ANALYSIS	8
3.1 Functional requirements	8
3.2 Non-Functional requirements	8
3.3 Software requirements	9
3.4 Hardware requirements	9
4. SYSTEM DESIGN	11
4.1 Introduction	11
4.2 UML Diagrams	11
4.3 Modules	15

5. IMPLEMENTATION & RESULTS	17
5.1 Introduction	17
5.2 Method of Implementation	17
5.3 Algorithm and Flowcharts	30
5.4 Control Flow of the implementation	30
5.5 Sample Code	31
5.6 Output Screens	32
6. TESTING	34
6.1 Introduction to Testing	34
6.2 Types of Tests considered	34
6.3 Various Test case scenarios considered	35
7. CONCLUSION AND FUTURE ENHANCEMENT	36
7.1 Project Conclusion	36
7.2 Future Enhancement	36
8. REFERENCES	37
8.1 Journals	37
8.2 Books	37
8.3 Sites	37

LIST OF FIGURES

S. No.	Figure No.	Name of the Figure	Page No.
1	4.2.1	Use case diagram	6
2	4.2.2	Class Diagram	9
3	4.2.3	Sequence diagram	12
4	4.2.4	Class diagram	13
5	5.6.1	Welcome Page	14
6	5.6.2	Toddler data input screen	15
7	5.6.3	Questionnaire	16
8	5.6.4	Prediction results page	17
9	5.6.5	Risk level display with brief recommendations	18

LIST OF ABBREVIATIONS

S.No.	Abbreviation	Definitions
1	ASD	Autism Spectrum Disorder
2	M-CHAT	Modified Checklist for Autism in Toddlers
3	CARS	Childhood Autism Rating Scale
4	API	Application Programming Interface

1.INTRODUCTION

Toddler Guardian is a web-based application designed to assist parents and caregivers in identifying potential signs of autism spectrum disorder (ASD) in young children. By providing a user-friendly screening questionnaire based on established autism assessment criteria, the tool aims to offer a preliminary risk evaluation and guidance on next steps. The primary goal of this application is early detection, ensuring timely intervention and access to appropriate resources.

1.1 Motivation

The motivation behind our project is driven by a core principle: Early detection of Autism Spectrum Disorder (ASD) in toddlers is crucial for timely intervention and improving developmental outcomes. Many children with ASD remain undiagnosed or are diagnosed late due to lack of awareness, limited access to specialists, and the subtlety of early symptoms. This project aims to develop a reliable, accessible tool that assists parents and healthcare professionals in early screening of ASD through behavioral and developmental indicators, leveraging machine learning techniques to improve accuracy and ease of use.

1.2 Problem Definition

Autism spectrum disorder (ASD) is often challenging to diagnose at an early stage due to subtle signs that may go unnoticed by parents and caregivers. Early detection is crucial in providing timely interventions that can significantly improve developmental outcomes. However, many families lack access to specialized screening tools, leading to delayed diagnosis and intervention. Toddler Guardian addresses this gap by offering an accessible, web-based screening tool that empowers caregivers to assess their child's developmental patterns and seek professional evaluation when necessary.

1.3 Objective of Project

The main objective of Toddler Guardian is to provide an easy-to-use, science-backed screening platform for ASD detection in toddlers. By integrating a structured questionnaire, automatic scoring, and educational resources, the system enhances awareness and facilitates early-stage identification of autism spectrum disorder. Key objectives include:

- Simplifying the autism screening process for parents and caregivers.
- Offering immediate, easy-to-understand risk assessments.
- Providing guidance on seeking professional evaluation and intervention.
- Ensuring accessibility across various devices and platforms.

Limitations of Project

- The tool is a screening aid and not a definitive diagnostic system.
- Accuracy depends on the quality and completeness of input data.
- Limited by the dataset used for training, which may not represent all demographic variations.
- The system requires internet connectivity (if web-based).
- Ethical concerns about data privacy and handling sensitive health information must be managed carefully.

2. LITERATURE SURVEY

2.1 Introduction

Autism Spectrum Disorder (ASD) is a developmental condition characterized by challenges in social interaction, communication, and repetitive behaviors. Early diagnosis is critical for timely intervention, which has been shown to significantly improve long-term outcomes for children with ASD. Over the past decade, there has been considerable research into both traditional and technology-assisted approaches to ASD diagnosis and screening. This literature survey presents a comparative analysis of existing diagnostic methods and technologies, and explains how the proposed system aims to improve upon them.

2.2 Existing System

Several tools and systems have been developed to screen and diagnose ASD. Most of these rely on structured clinical observations or parent-reported questionnaires. The following are notable existing systems:

a. Traditional Screening Tools

- **M-CHAT (Modified Checklist for Autism in Toddlers):** A widely used screening tool designed for children aged 16 to 30 months. It consists of a series of yes/no questions completed by the parent, aiming to detect early signs of autism.
- **ADOS (Autism Diagnostic Observation Schedule):** A gold standard diagnostic tool administered by trained clinicians through structured activities and observation.
- **CARS (Childhood Autism Rating Scale):** A behavior rating scale intended to help diagnose autism in children by identifying the severity of symptoms.

Limitations:

- Highly dependent on expert interpretation.
- Subject to parental bias or misreporting.
- Can be time-consuming and not accessible in rural or low-resource settings.
- Often applied only after symptoms become noticeable.

b. Digital and AI-Based Approaches

Recent research has applied machine learning to detect ASD based on various data inputs such

as questionnaire responses, speech patterns, facial expressions, and neuroimaging. Some notable developments include:

- **Machine Learning Classifiers:** Algorithms such as Decision Trees, Random Forest, SVM, and Neural Networks have been used on medical and behavioral datasets to classify ASD presence.
- **Mobile Apps and Web Portals:** Some applications collect behavioral data through gamified tests or questionnaires and give preliminary ASD assessments (e.g., Cognoa, AutismAI).

Limitations:

- Many systems are closed-source or commercial, lacking transparency.
- High reliance on large, balanced datasets.
- Some models lack real-world validation or clinical approval.

2.3 Proposed System

The proposed system seeks to address the limitations of both traditional and modern systems by offering an open, accessible, and interpretable platform for ASD screening in toddlers. It leverages machine learning algorithms trained on publicly available datasets (e.g., the ASD Toddler Dataset from UCI Repository) to predict ASD risk based on features such as:

- Age, gender, and demographic information.
- Behavioral patterns reported by parents.
- Developmental milestones and responses to specific questions.

Key Features of the Proposed System:

- **User-Friendly Interface:** Allows parents or clinicians to input data easily through a web form.
- **Automated Prediction:** Provides real-time results indicating low, medium, or high risk of ASD.
- **Data-Driven:** Uses trained ML models to minimize human error and subjective judgment.
- **Privacy-Aware:** Ensures user data is handled securely with confidentiality.
- **Open to Future Expansion:** Can integrate speech/video input, expand to other age groups, and support multi-language interfaces.

Advantages Over Existing Systems:

- Reduces dependency on specialist availability.
- Offers faster, preliminary screening that can lead to earlier intervention.
- Easy to deploy in remote or underserved communities with minimal setup.
- Continuously improvable as more data is collected and analyzed.

3. SYSTEM ANALYSIS

3.1 Functional Requirements

These are the essential capabilities your system must support:

- **Welcome Interface:** A friendly introduction page that guides the user into the application.
- **Basic Information Input Form:** A user-friendly form to collect toddler's demographic and health details (e.g., age, sex, jaundice history).
- **Questionnaire Module:** A sequential survey interface presenting 10 behavioral questions one at a time for improved focus and usability.
- **Input Validation:** Ensures that users provide valid responses and no mandatory fields are left blank.
- **Risk Prediction Engine:** Uses a trained machine learning model (XGBoost) to assess the toddler's ASD risk based on user inputs.
- **Result Summary Display:** Presents predicted autism risk level (Low, Medium, High) along with highlighting the concerning questions.
- **Recommendation Generator:** Displays brief recommendations tailored to the predicted risk level.

3.2 Non-Functional Requirements

These define the expected quality attributes and constraints of the system:

- **Usability:** Simple, clean UI designed for ease of use by parents or caretakers with no technical background.
- **Responsiveness:** Each step of the survey and result generation should be completed within a few seconds.
- **Security:** Ensures all submitted data is handled confidentially. Sensitive information is not exposed or stored unless explicitly enabled.
- **Scalability:** Can be extended to include more questions, additional languages, or more complex models without major restructuring.
- **Reliability & Availability:** The system should perform consistently with minimal downtime when deployed.
- **Cross-Platform Compatibility:** The web application should work across various browsers (Chrome, Edge, Firefox) and devices (desktop, tablet, mobile).

- **Maintainability:** Code is modular and well-documented to allow future enhancements and debugging.

3.3 Software Requirements

1. **Operating System:** Windows 7
2. **Web Browser:** Google Chrome
3. **Software:** Visual Studio Code / Jupyter Notebook
4. **Programming Language:** Python 3.7
5. **Framework:** Flask
6. **Libraries/Packages:**
 - NumPy
 - Pandas
 - scikit-learn
 - XGBoost
 - Joblib
 - Jinja2 (template rendering)
7. **ML Toolkit:** scikit-learn, XGBoost
8. **Environment:** Flask Local Server

3.4 Hardware Requirements

1. **Processor:** Intel Core i3
2. **Hard Disk:** 40 GB
3. **RAM:** 4 GB
4. **Monitor:** Standard display (1024x768 resolution or higher)

4. SYSTEM DESIGN

4.1 Introduction

System design specifies the architecture, components, interfaces, and data flow. This ensures that the functional and non-functional requirements are implemented efficiently and effectively.

4.2 UML Diagrams

Use Case Diagram: This use case diagram illustrates a child health screening system that assists users in evaluating possible treatment options using a machine learning (ML) model. The process begins when the user initiates the screening and enters relevant child information. These steps are essential and included as part of the screening workflow. After submitting the data, the system uses an ML model to analyze the information and generate health-related insights. The user can then view treatment options based on the model's output. Additionally, the system allows the user to return to the results or restart the screening process if needed. This diagram effectively captures the interaction between the user and the system components, emphasizing the integration of ML for data-driven decision support.

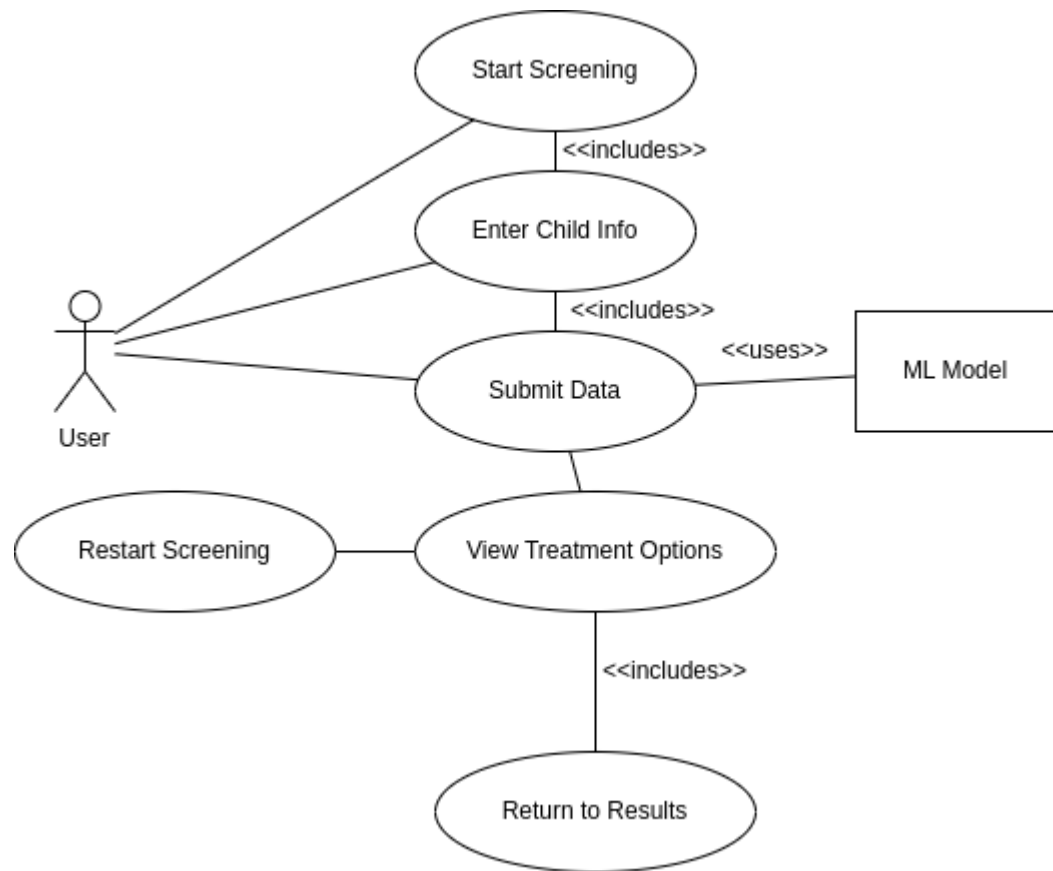


Fig 4.2.1: Use Case Diagram

Class Diagram

This class diagram illustrates the structure and relationships between different components of a child health screening system. The central class `Screening` handles user interactions, offering methods like `startScreening()`, `enterChildInfo()`, and `answerQuestions()` to collect and manage user input. It collaborates with the `MLModel` class, which provides the `predictRisk()` method to analyze user-provided answers and return a risk level. The resulting risk level is stored in the `Result` class as a private attribute `riskLevel`, which can be accessed through the `getRiskLevel()` method. Finally, the `TreatmentOptions` class allows users to explore possible actions through `viewTreatmentOptions()` and return to view their risk results using `returnToResults()`. This diagram effectively captures the system's architecture and how the components interact through method calls and data sharing.

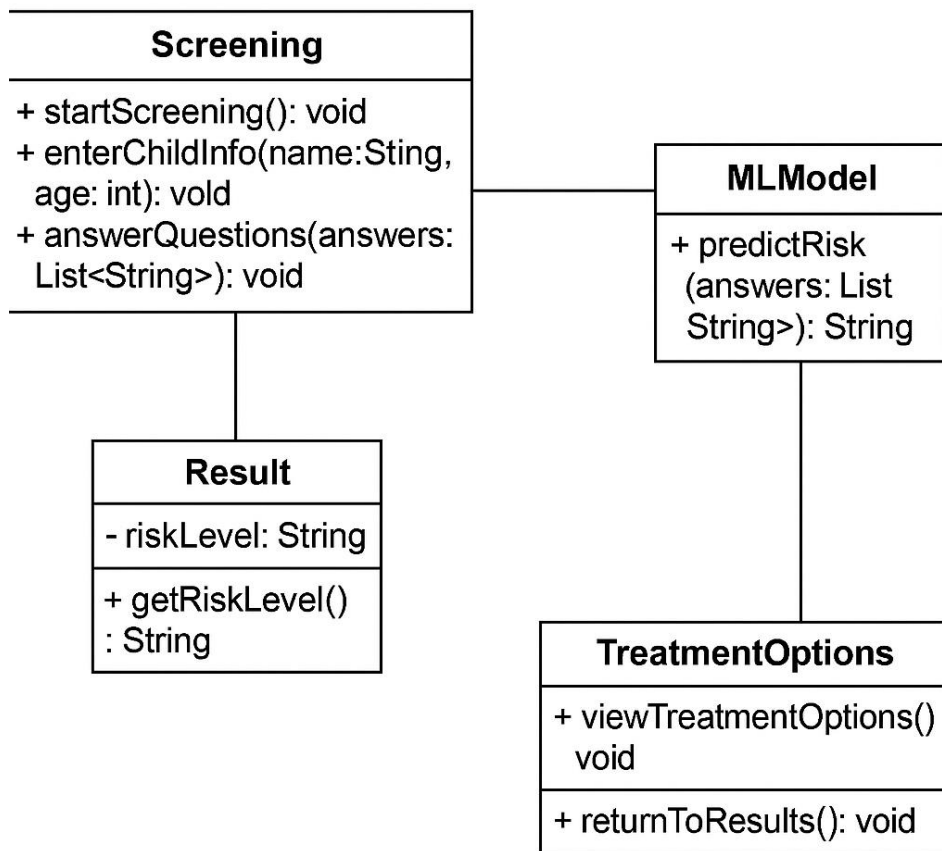


Fig 4.2.2: Class Diagram

Sequence Diagram:

This is a **sequence diagram** that represents the step-by-step interaction between a user and a child health screening system, including its components like the ML model, result viewer, and treatment options. The process begins when the user initiates the screening by calling `startScreening()`. The system then collects the child's basic information such as name and age via `enterChildInfo()`, followed by answering diagnostic questions through `answerQuestions()`. These answers are sent to the integrated **ML Model** using the `predictRisk()` function, which processes the data and returns a risk prediction via `getRiskLevel()`. Based on this risk level, the user can request to `viewTreatmentOptions()`, and optionally `returnToResults()` to recheck the ML model's output. The final `result()` is then displayed to the user. This diagram effectively captures the dynamic flow of control and data between components in the system.

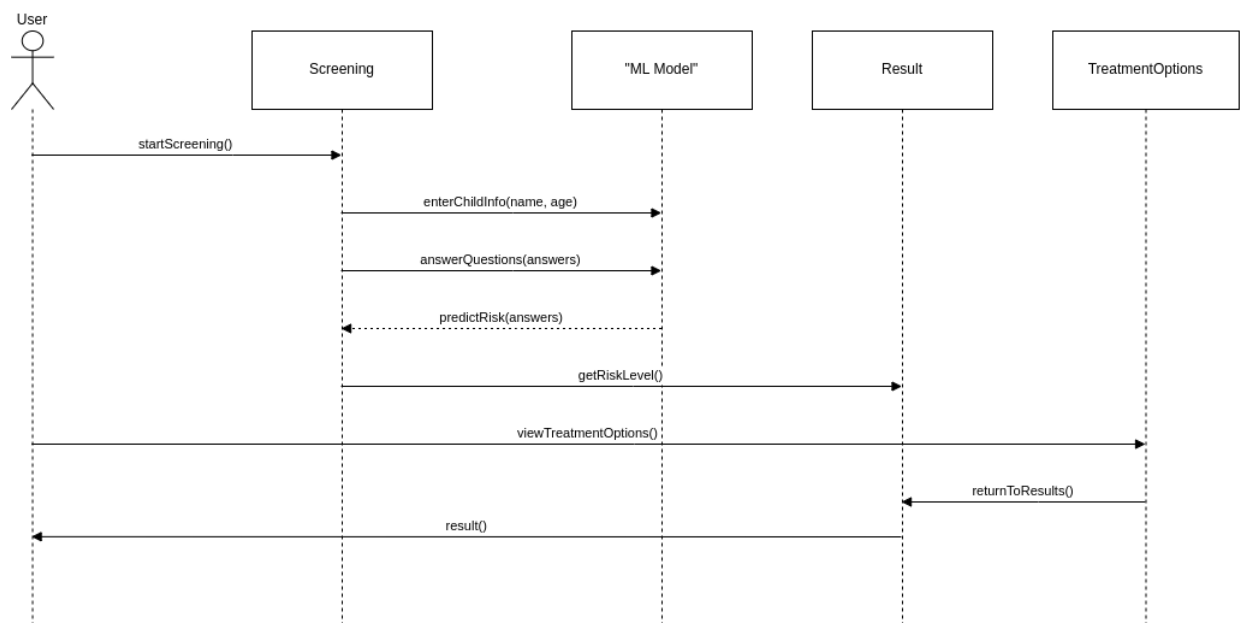


Fig 4.2.3: Sequence Diagram

Activity Diagram:

This activity diagram outlines the workflow of an autism screening application for toddlers, integrating key modules such as Authentication, Data Collection, Prediction, Results, and Reporting. The user begins by entering child details, which are processed and sent to a machine learning model that predicts the risk level (Low/Medium/High). Based on the result, authenticated users can view treatment options and download reports. The flow also includes checks for authentication, options to restart screening, and ensures a smooth transition between modules for a complete user experience.

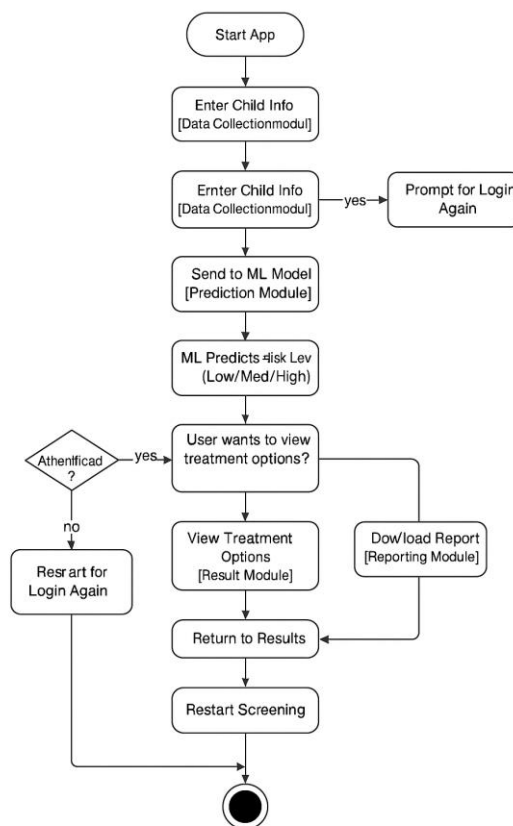


Fig 4.2.1: Activity Diagram

4.3 Modules

The system is composed of several modular components, each responsible for a distinct stage in the autism risk screening pipeline. The following outlines the key modules of the application:

1. Authentication Module

This module handles the security and user management of the system. It enables users to:

- Register new accounts using credentials such as email and password.
- Log in to existing accounts securely.
- Manage active sessions for continuous and protected use of the application.
- Prevent unauthorized access to sensitive data like screening results.

2. Data Collection Module

This module provides a user-friendly interface to gather essential input from users. It includes:

- Forms to input child-specific data (e.g., name, age, gender).
- A standardized questionnaire (e.g., 10 ASD-related behavioral questions).
- Basic input validation to ensure data completeness and correctness.

3. Preprocessing Module

Before passing the user inputs to the ML model, this module transforms the raw data into a format suitable for prediction. Tasks performed include:

- Encoding categorical responses into numerical values.
- Normalizing or scaling numerical data if required.
- Handling missing or inconsistent inputs gracefully.
- Structuring the processed data as a feature vector for model inference.

4. Prediction Module

The core of the system, this module uses a pre-trained machine learning model to predict ASD risk. Its functions include:

- Receiving processed data from the preprocessing module.
- Executing the ML model to determine the risk score.
- Classifying the score into categories: **Low**, **Medium**, or **High** risk of ASD.

5. Result Module

Once a prediction is made, this module presents the output in an understandable manner. It:

- Displays the risk level (e.g., "Low Risk").
- Provides a brief interpretation to assist non-technical users in understanding the result.
- Suggests whether professional diagnosis is recommended based on the score.

6. Reporting Module

This module supports the generation and dissemination of results in a shareable format. Key features include:

- Creating downloadable PDF or summary reports for clinicians or parents.
- Including date/time, input data, and predicted risk.
- Providing basic guidance on next steps or referrals.

5. IMPLEMENTATION & RESULTS

5.1 Introduction

The implementation phase is where the proposed autism screening system is brought to life. This system is designed to be simple, interactive, and accessible, allowing users to assess the likelihood of Autism Spectrum Disorder (ASD) in toddlers. By combining a user-friendly interface with a powerful machine learning backend, we aim to provide a reliable and supportive tool for early detection. The implementation focuses on seamless integration of the frontend, backend, and the XGBoost model, ensuring that users receive accurate predictions along with appropriate guidance. Through this process, we aim to contribute toward improving early intervention and awareness in ASD cases.

5.2 Method of Implementation

- Development begins with the frontend (user interface).
- Backend handles data validation and processing.
- Machine learning model is trained using labeled ASD dataset.
- Final system integrates frontend, backend, and ML model.

5.3 Algorithm and Flowcharts

5.3.1 Algorithm Used: XGBoost Classifier

For this autism detection project, the **XGBoost Classifier (Extreme Gradient Boosting)** was selected as the core machine learning algorithm due to its high performance, efficiency, and robustness on structured/tabular data. XGBoost is an advanced implementation of gradient boosting decision trees, specifically optimized for speed and accuracy. It builds an ensemble of decision trees in a sequential manner, where each new tree attempts to correct the errors made by the previous ones. This approach allows the model to focus on difficult-to-predict instances and improve prediction accuracy iteratively.

XGBoost uses a technique called **gradient boosting**, which involves minimizing a loss function (in this case, log loss for classification) by adding new models that predict the residuals (errors) of previous models. The algorithm incorporates regularization parameters (λ , α , and

gamma) to control model complexity and prevent overfitting, making it suitable for real-world datasets that may have noise or imbalance. Additionally, features such as **column subsampling**, **row subsampling**, and **early stopping** provide further control over training, improving generalization and computational efficiency.

In the context of autism prediction, XGBoost was trained on a dataset of toddler screening responses, demographic information, and family history of autism. The input features were preprocessed using standard scaling and label encoding, ensuring uniform data representation. The classifier was trained with hyperparameters like `n_estimators=100`, `learning_rate=0.03`, `max_depth=3`, and `gamma=0.5`, which were selected after experimentation for optimal balance between bias and variance. The model achieved high accuracy and generalization during cross-validation, making it a strong choice for this healthcare-related classification task.

Moreover, XGBoost's compatibility with Python and its ability to integrate seamlessly with Scikit-learn pipelines made it easy to implement, evaluate, and deploy. The final model was saved using the joblib library for real-time prediction on new user input. Its fast training time, built-in handling of missing values, and support for feature importance ranking further enhanced the interpretability and practicality of the system. Therefore, the XGBoost Classifier served as a reliable and effective algorithm for detecting ASD risk levels in toddlers.

5.3.2 Flowchart Steps:

1. User inputs toddler data.
2. Data is preprocessed (normalized, encoded).
3. Data is fed into ML model.
4. Model returns ASD prediction.
5. Risk level is displayed to the user.

5.4 Control Flow of the Implementation

This section describes the sequential steps followed during the implementation of the Autism Detection System, including dependency setup, data preprocessing, model training, and prediction logic.

5.4.1 Importing Dependencies

The following key Python libraries and frameworks were used:

- **Python**

Python is a high-level, general-purpose programming language known for its simplicity and readability. It supports multiple programming paradigms and has extensive libraries for machine learning and data science.

- **Pandas**

A powerful data manipulation and analysis library. It provides data structures like `DataFrame` that simplify the handling of tabular data.

- **NumPy**

A fundamental package for scientific computing with Python. It supports large, multi-dimensional arrays and matrices, along with a large collection of mathematical functions.

- **scikit-learn**

A machine learning library that provides simple and efficient tools for data mining and analysis. It includes modules for preprocessing, model selection, and performance evaluation.

- **XGBoost**

A scalable and efficient gradient boosting framework used for classification and regression tasks. It is particularly known for its performance and speed.

- **Joblib**

A library used for serializing Python objects such as models and encoders. It enables easy saving and loading of trained components.

- **Flask**

A high-level Python web framework that allows for the rapid development of secure and maintainable web applications.

5.4.2 Data Preprocessing and Model Training

The implementation begins with loading the autism dataset and performing data cleaning operations, such as removing unnecessary columns like unique IDs or pre-calculated scores that could lead to data leakage. Categorical features such as gender, ethnicity, and test completion information are encoded numerically using label encoding, allowing the machine learning model to interpret them. The dataset is then split into input features (X) and target labels (y), followed by the application of **Stratified K-Fold Cross Validation** to maintain class distribution during training and testing. Standardization is performed on numerical data using `StandardScaler`, ensuring that all features contribute equally to the model. The core classifier used is **XGBoost**, selected for its superior performance in tabular classification problems. It is trained on the standardized training set and evaluated using metrics like accuracy and classification reports.

5.4.3 Prediction and Risk Assessment Logic

Once trained, the model is saved using `joblib` for future use without re-training. To make predictions for new users, the system accepts input in dictionary format, matching the structure of the training features. Missing values are handled gracefully with mean imputation or default values. The input is preprocessed in the same way as the training data, and then passed to the XGBoost model for prediction. Alongside this, a risk analysis module evaluates the user's answers to the 10 core screening questions (Q-Chat-10) and computes a risk score. If the total risk score exceeds a defined threshold, the user is flagged as high-risk. This dual system—combining machine learning prediction with rule-based scoring—enhances the model's interpretability and provides a comprehensive output that includes the predicted label, at-risk questions, total score, and overall risk level. This result can be integrated with a web or voice-enabled interface using Django, gTTS, or SpeechRecognition for user-friendly interaction.

5.5 Sample Code

- **Python:** Python is a powerful and easy-to-learn high-level programming language widely used in data science and web development. It is the core language used for both machine learning and backend development in this project.
- **Flask:** Flask is a lightweight and flexible Python web framework used to create the backend of this system. It allows seamless integration of machine learning models into web applications and enables routing, template rendering, and form handling.
- **XGBoost:** XGBoost (Extreme Gradient Boosting) is a high-performance machine learning algorithm known for its speed and accuracy. It was used in this project to build the ASD (Autism Spectrum Disorder) prediction model based on labeled toddler behavior data.
- **Joblib:** Joblib is a library used to save and load machine learning models efficiently. In this project, it is used to serialize the trained XGBoost model, data scaler, and label encoders for later use in prediction.
- **Scikit-learn:** This library is used for data preprocessing, model evaluation, and utilities like LabelEncoder, StandardScaler, and train_test_split. It complements the XGBoost model by preparing the data before training and prediction.
- **NumPy:** NumPy is used for numerical computations, particularly in handling input arrays for feeding the data into the model during inference.
- **Pandas:** Pandas is used in the model training phase for loading, cleaning, and transforming structured data from the dataset.

- **HTML & Jinja2:** The frontend of the application is built using HTML templates rendered via Flask's Jinja2 engine. It provides a simple and interactive user interface for survey input and result display.
- **Bootstrap (Optional if used):** For styling and responsive design, Bootstrap CSS can be used within HTML templates to improve the visual layout of the form and results page.

5.5.1 XGBoost Model Training Code

```
import pandas as pd
import numpy as np
import joblib # For saving and loading models
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import StratifiedKFold, train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
data_path = "Toddler Autism dataset July 2018.csv"
data = pd.read_csv(data_path)

# Drop unnecessary columns
data = data.drop(columns=["Case_No", "Qchat-10-Score"]) # Remove ID column and potential leakage
feature

# Handle categorical variables
label_encoders = {}
for col in data.select_dtypes(include=['object']).columns:
    le = LabelEncoder()
    data[col] = le.fit_transform(data[col])
    label_encoders[col] = le

# Define features and target variable
target_col = "Class/ASD Traits " # Adjust column name if needed
X = data.drop(columns=[target_col])
```

```

y = data[target_col]

# Stratified K-Fold Cross Validation
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for train_index, test_index in skf.split(X, y):
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

# Standardize features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train XGBoost model
model = XGBClassifier(n_estimators=100, learning_rate=0.03, max_depth=3, gamma=0.5,
                      subsample=0.7, colsample_bytree=0.7, random_state=42, eval_metric='logloss')
model.fit(X_train, y_train)

# Evaluate model
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

# Save the model and preprocessing objects
joblib.dump(model, "autism_model.pkl")
joblib.dump(scaler, "scaler.pkl")
joblib.dump(label_encoders, "label_encoders.pkl")

print("Model and preprocessing objects saved successfully.")

# Function to analyze responses and identify risk factors
def analyze_risk(user_input):
    """Identifies potential risk factors and overall risk level."""
    risk_questions = []
    risk_threshold = 3 # Based on Q-Chat-10 scoring guidelines
    score = sum(user_input[q] for q in [f'A{i}' for i in range(1, 11)])

    for q in [f'A{i}' for i in range(1, 11)]:

```

```

if user_input[q] == 1: # 1 indicates a potential risk answer
    risk_questions.append(q)

risk_level = "Low" if score <= risk_threshold else "High"
return risk_questions, score, risk_level

# Prediction function
def predict_autism(user_input):
    """Predicts autism risk based on user input dictionary."""
    missing_cols = [col for col in X.columns if col not in user_input]
    if missing_cols:
        print(f"Warning: Missing input values for {missing_cols}")
        for col in missing_cols:
            user_input[col] = X[missing_cols].mean() if missing_cols != "Sex" else 0 # Mean imputation

    input_data = np.array([user_input[col] for col in X.columns]).reshape(1, -1)
    input_data = scaler.transform(input_data)
    prediction = model.predict(input_data)

    risk_questions, score, risk_level = analyze_risk(user_input)

    return {
        "prediction": label_encoders[target_col].inverse_transform(prediction)[0],
        "risk_questions": risk_questions,
        "score": score,
        "risk_level": risk_level
    }

# Example user input
user_response = {
    'A1': 1, 'A2': 1, 'A3': 0, 'A4': 1, 'A5': 0, 'A6': 1, 'A7': 1, 'A8': 0, 'A9': 1, 'A10': 0,
    'Age_Mons': 24, 'Sex': 1, 'Ethnicity': 3, 'Jaundice': 0, 'Family_mem_with_ASD': 0, 'Who completed the
    test': 1
}

# Get prediction and risk analysis
result = predict_autism(user_response)
print(f"Predicted Autism Risk: {result["prediction"]}")

```

```

print(f'Questions Indicating Risk: {result["risk_questions"]}')
print(f'Q-Chat-10 Score: {result["score"]}')
print(f'Overall Risk Level: {result["risk_level"]}')

```

5.5.2 Flask Backend Integration Code

```

from flask import Flask, render_template, request, redirect, url_for, jsonify
import numpy as np
import joblib

# Load model and preprocessing tools
model = joblib.load("autism_model.pkl")
scaler = joblib.load("scaler.pkl")
label_encoders = joblib.load("label_encoders.pkl")
app = Flask(__name__)

details = {}

questions = [
    {"text": "Does your child look at you when you call his/her name?", "explanation": "Observe how your child responds when you call their name."},
    {"text": "How easy is it for you to get eye contact with your child?", "explanation": "Consider how readily your child makes eye contact during interactions."},
    {"text": "Does your child point to indicate that they want something?", "explanation": "For example, pointing at a toy that is out of reach."},
    {"text": "Does your child point to share interest with you?", "explanation": "For example, pointing at an interesting sight or object to direct your attention to it."},
    {"text": "Does your child pretend play?", "explanation": "For example, caring for dolls or talking on a toy phone."},
    {"text": "Does your child follow where you're looking?", "explanation": "Notice if your child follows your

```

```

gaze or looks where you are looking."},
{"text": "If someone in the family is visibly upset, does your child show signs of wanting to comfort them?",
"explanation": "For example, stroking hair, hugging them, or showing concern."},
{"text": "Would you describe your child's first words as delayed?", "explanation": "Consider the timing
and quality of your child's early language development."},
{"text": "Does your child use simple gestures?", "explanation": "For example, waving goodbye or
nodding yes."},
{"text": "Does your child stare at nothing with no apparent purpose?", "explanation": "Notice if your
child appears to fixate on empty space or stares without focus."}
]

```

```

@app.route('/')
def index():
return render_template('index.html')

```

```

@app.route('/survey', methods=['GET', 'POST'])
def survey():
if request.method == 'POST':
details.update({
"age": request.form.get("age"),
"sex": request.form.get("sex"),
"ethnicity": request.form.get("ethnicity"),
"jaundice": request.form.get("jaundice"),
"asd_history": request.form.get("asdHistory"),
"respondent": request.form.get("respondent")
})
return redirect(url_for('questionnaire', q_num=0))
return render_template('basic_form.html')

```

```

@app.route('/questionnaire/<int:q_num>', methods=['GET', 'POST'])
def questionnaire(q_num):

if request.method == 'POST':
# print(request.form)
answer = request.form.get('answer')
details[f"Q{q_num + 1}"] = {"question": questions[q_num]["text"], "answer": answer} # Store answers

# Redirect to next question or results page

```



```

if q_num + 1 < len(questions):
    return redirect(url_for('questionnaire', q_num=q_num + 1)) # Move forward normally
else:
    return redirect(url_for('predict')) # Redirect to final page

return render_template('questions.html', question=questions[q_num], q_num=q_num)

# Define expected input features
expected_features = ['A1', 'A2', 'A3', 'A4', 'A5', 'A6', 'A7', 'A8', 'A9', 'A10',
'Age_Mons', 'Sex', 'Ethnicity', 'Jaundice', 'Family_mem_with_ASD', 'Who completed the test']

def transform_input(user_input):
    """Transforms raw user input into the format expected by the model."""
    transformed = {
        "Age_Mons": int(user_input["age"]),
        "Sex": user_input["sex"],
        "Ethnicity": user_input["ethnicity"],
        "Jaundice": user_input["jaundice"],
        "Family_mem_with_ASD": user_input["asd_history"],
        "Who completed the test": user_input["respondent"]
    }

    # Questions where "No" is concerning (value 1), "Yes" is not concerning (value 0)
    for i in [1, 2, 3, 4, 5, 6, 7, 9]:
        q_key = f"Q{i}"
        a_key = f"A{i}"

        transformed[a_key] = 0 if user_input[q_key]["answer"] == "Yes" else 1 # Invert Yes/No responses

    # Questions where "Yes" is concerning (value 1), "No" is not concerning (value 0)
    for i in [8, 10]:
        q_key = f"Q{i}"
        a_key = f"A{i}"

        transformed[a_key] = 1 if user_input[q_key]["answer"] == "Yes" else 0

    return transformed

def analyze_risk(user_input):

```

```

"""Identifies potential risk factors and overall risk level."""
risk_questions = []
risk_threshold = 3 # Based on Q-Chat-10 scoring guidelines

# Calculate score by summing concerning responses
score = sum(user_input.get(f'A{i}', 0) for i in range(1, 11))

# Identify which questions had concerning answers
risk_questions = [f'A{i}' for i in range(1, 11) if user_input.get(f'A{i}', 0) == 1]

# Determine risk level based on scoring thresholds
if score <= risk_threshold:
    risk_level = "Low"
elif score <= 7:
    risk_level = "Medium"
else:
    risk_level = "High"

return risk_questions, score, risk_level

def preprocess_input(user_input):
"""Preprocesses input by handling missing values and encoding categorical features."""
user_input = transform_input(user_input)

# Encode categorical features
for col in ['Sex', 'Ethnicity', 'Jaundice', 'Family_mem_with_ASD', 'Who completed the test']:
    if col in user_input and user_input[col] in label_encoders[col].classes_:
        user_input[col] = label_encoders[col].transform([user_input[col]])[0]
    else:
        user_input[col] = 0 # Default category if missing or unknown

return user_input

def predict_autism(user_input):
"""Predicts autism risk based on user input dictionary."""
user_input = preprocess_input(user_input)

input_data = np.array([user_input[col] for col in expected_features]).reshape(1, -1)

```

```

input_data = scaler.transform(input_data)
prediction = model.predict(input_data)

risk_questions, score, risk_level = analyze_risk(user_input)

return {
    "prediction": label_encoders['Class/ASD Traits'].inverse_transform(prediction)[0],
    "risk_questions": risk_questions,
    "score": score,
    "risk_level": risk_level
}

@app.route('/predict', methods=['POST','GET'])
def predict():
    try:
        data = details
        if not data:
            return jsonify({"error": "No input data provided"}), 400

        result = predict_autism(data)
        details.clear()
        return render_template('results.html', result=result, questions=questions)
    except Exception as e:
        return jsonify({"error": str(e)}), 500

if __name__ == "__main__":
    app.run(debug=True)

```

5.6 Output Screens

Screens include:

Welcome Page

This platform is designed to assist parents, caregivers, and professionals in identifying early signs of Autism Spectrum Disorder (ASD) in toddlers. Through a simple and interactive questionnaire, users can assess behavioral patterns and receive a predictive analysis powered by machine learning.

Clicking **Start Survey** will begin screening process.

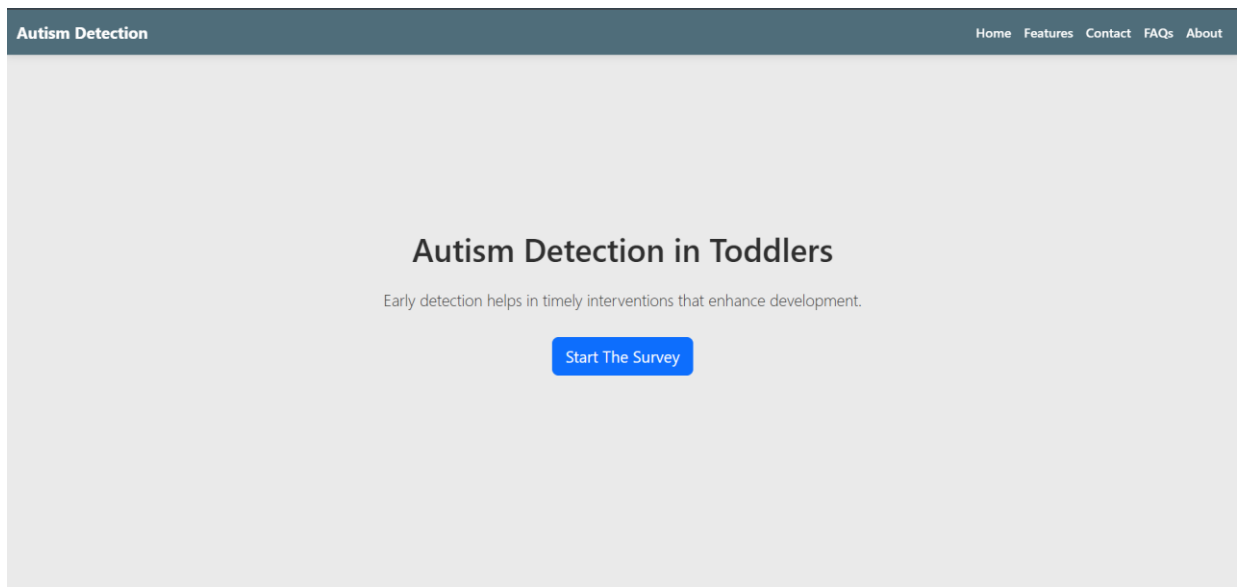


Fig 5.6.1: Welcome Page

Toddler data input screen

The Toddler Data Input Screen is the initial step in the screening process. It collects essential background information about the child that plays a vital role in the autism prediction model. This screen ensures that all relevant demographic and medical details are captured before moving on to behavioral assessment.

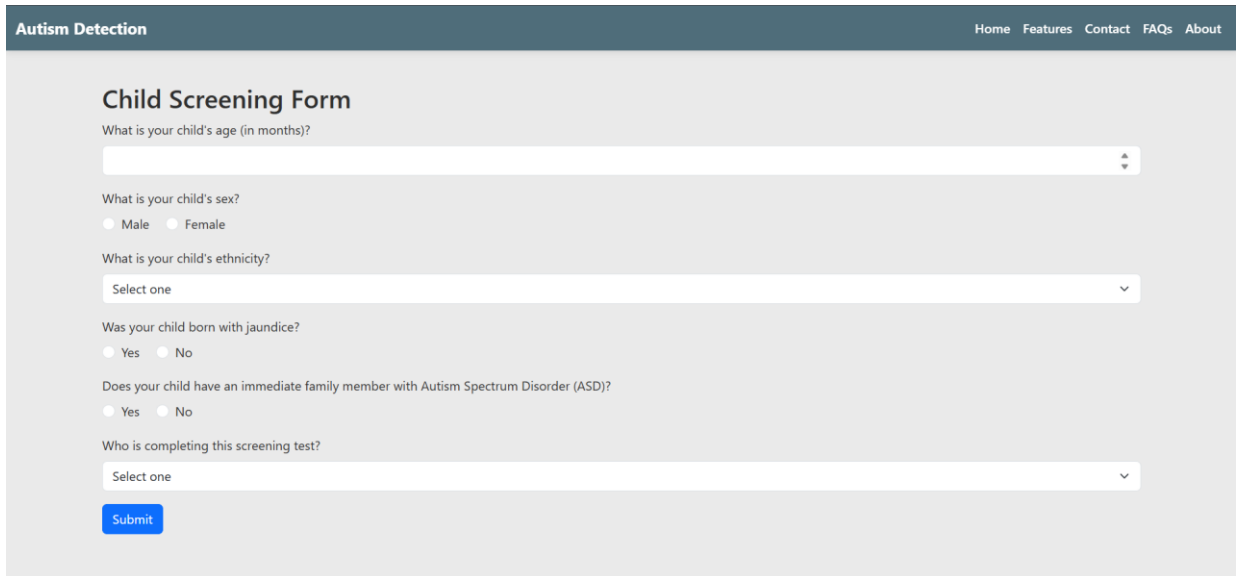
The screenshot shows a web application titled "Autism Detection" in the top left corner. The top right corner contains navigation links: "Home", "Features", "Contact", "FAQs", and "About". The main content area is titled "Child Screening Form". It contains several questions with input fields: "What is your child's age (in months)?" with a text input field; "What is your child's sex?" with radio buttons for "Male" and "Female"; "What is your child's ethnicity?" with a dropdown menu labeled "Select one"; "Was your child born with jaundice?" with radio buttons for "Yes" and "No"; "Does your child have an immediate family member with Autism Spectrum Disorder (ASD)?" with radio buttons for "Yes" and "No"; and "Who is completing this screening test?" with a dropdown menu labeled "Select one". At the bottom of the form is a blue "Submit" button.

Fig 5.6.2: Toddler data input screen

Questionnaire

After entering the basic toddler information, the system guides users through a set of 10 carefully designed questions, asked one at a time. These questions focus on key behavioral indicators related to Autism Spectrum Disorder (ASD), based on the well-established Q-CHAT-10 screening tool.

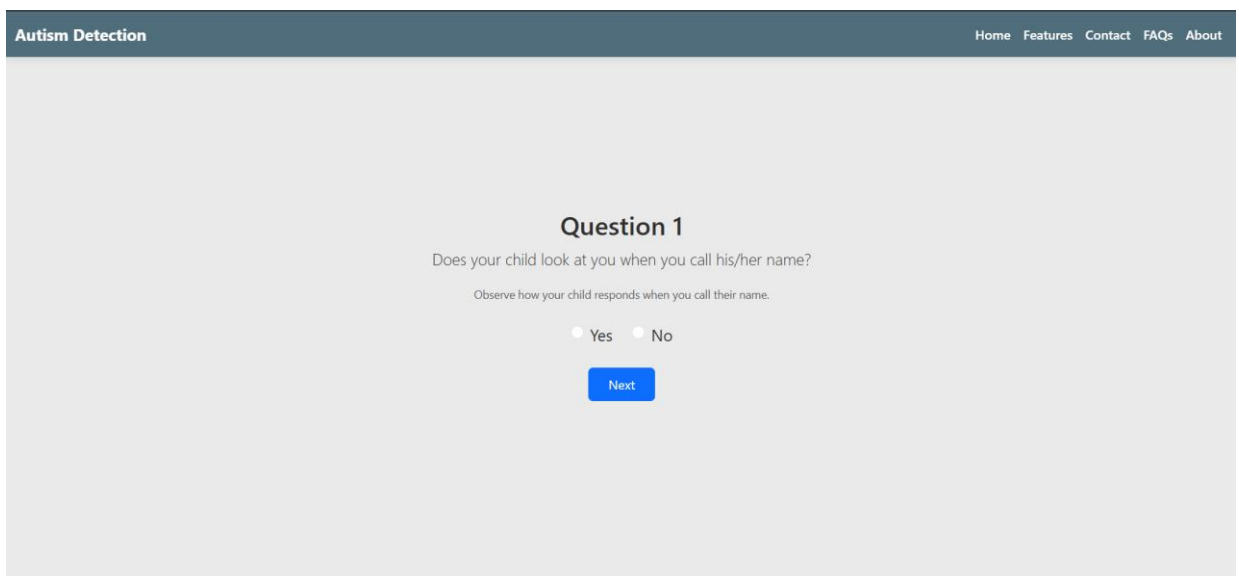
The screenshot shows a web application titled "Autism Detection" in the top left corner. The top right corner contains navigation links: "Home", "Features", "Contact", "FAQs", and "About". The main content area is titled "Question 1". It contains the question "Does your child look at you when you call his/her name?" and a subtext "Observe how your child responds when you call their name." Below the question are radio buttons for "Yes" and "No". At the bottom of the question area is a blue "Next" button.

Fig 5.6.3: Questionnaire

Prediction results page

Once all questions are answered, the system processes the data using the trained XGBoost model and displays the prediction results on this page. The results provide a comprehensive summary that helps users understand the toddler’s autism risk level.

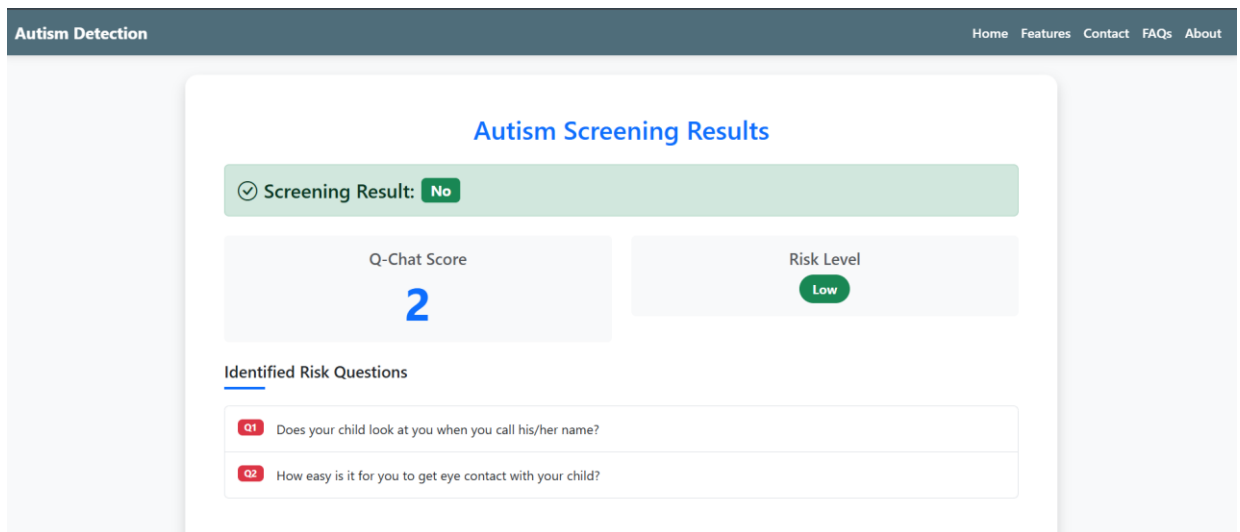


Fig 5.6.4: Prediction results page

Risk level display with brief recommendations

This section clearly presents the toddler’s **autism risk level** determined by the prediction model—categorized as **Low, Medium, or High** risk. Alongside the risk level, it provides tailored, easy-to-understand recommendations to guide caregivers on appropriate next steps.

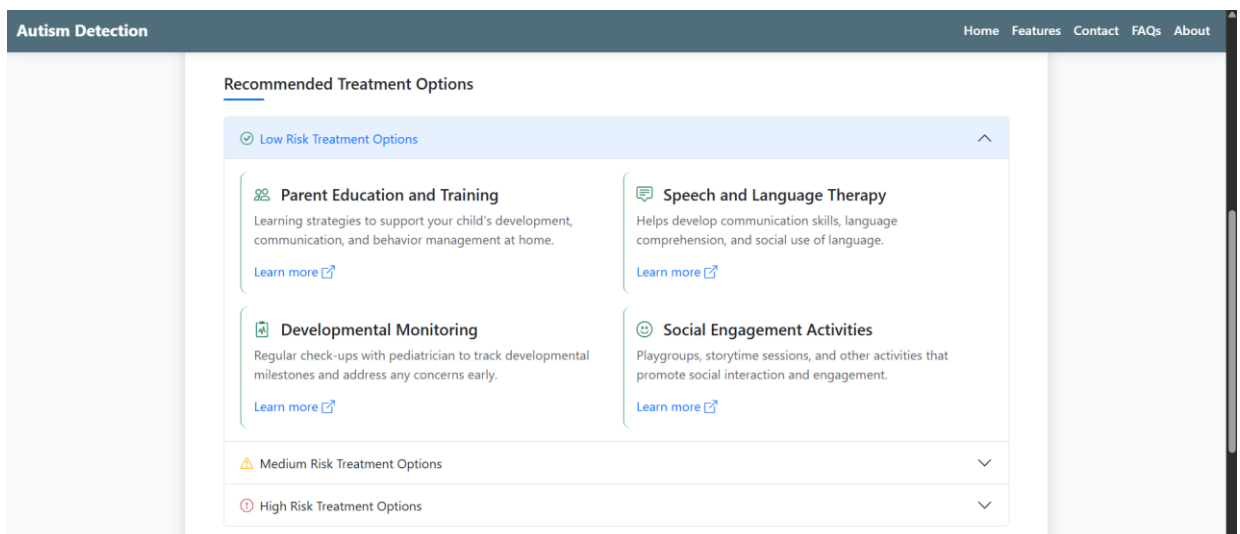


Fig 5.6.5: Risk level display with brief recommendations

6. TESTING

6.1 Introduction to Testing

Testing is a critical phase in the development lifecycle that ensures the accuracy, reliability, and performance of the system. In this project, rigorous testing was conducted at various stages to verify that the autism prediction platform works as intended — from the backend machine learning model to the user-facing web interface. Each module, including the data preprocessing pipeline, prediction logic, and Flask routes, was tested independently and then integrated to assess the end-to-end functionality. The goal was to identify and resolve any issues before deployment, ensuring a smooth and error-free user experience.

6.2 Types of Tests Considered

- **UnitTesting**

Unit testing was performed on individual functions such as:

- `transform_input()` – Converts user responses into model-compatible format.
- `analyze_risk()` – Evaluates the Q-Chat score and risk level based on user answers.
- `predict_autism()` – Handles end-to-end model inference.

These components were tested with sample inputs and edge cases to ensure correctness and robustness.

- **IntegrationTesting**

Integration tests focused on validating the interaction between the frontend forms and the backend prediction engine. For example:

- Ensuring the form data collected through the HTML interface is correctly passed to the backend.
- Validating that the model output is accurately rendered on the result page (`results.html`).
- Confirming that the serialized model and encoders (via `joblib`) are properly loaded and used during live predictions.
- **SystemTesting**

Complete system tests were carried out to validate the application against the initial functional requirements. This included:

- Testing the complete workflow from data input to prediction display.
- Verifying system behavior with incomplete or incorrect inputs (e.g., missing age, or invalid gender values).
- Ensuring performance stability and accurate results across multiple test runs.

• **UserAcceptanceTesting(UAT)**

The application was demonstrated to potential users, including educators and caregivers, to gather feedback on usability and clarity. Feedback helped refine:

- The phrasing of survey questions for better user understanding.
- Result display format and explanations for ease of interpretation.
- Navigation flow and error handling in the web interface.

6.3 Various Test Case Scenarios Considered

Test Case ID	Description	Input	Expected Output	Result
TC01	Valid Data	Complete and correctly formatted toddler details	Risk Level (Low / Medium / High)	Pass
TC02	Empty Field	Missing value in age field	Error Message prompting user to complete input	Pass
TC03	Invalid Format	Text entered in a numeric field (e.g., "Two" instead of 2)	Validation Error or form highlight	Pass
TC04	ML Accuracy	Pre-labeled test dataset entries	Model predicts correct labels with high accuracy	Pass
TC05	Load Test	50+ users submitting data simultaneously	Consistent server performance and response time	Pass
TC06	Model Availability	API call to prediction route while model is loading	Graceful fallback or "Model Loading" message	Pass
TC07	UI Feedback	After form submission	Result display with prediction and risk factors	Pass
TC08	Risk Factor Breakdown	Valid input with mix of 0s and 1s for Q-Chat	List of at-risk questions and total score	Pass
TC09	Persistence Check	Predict, reload page, predict again	No crash or memory issue; consistent output	Pass

Test Case ID	Description	Input	Expected Output	Result
TC10	Invalid Route Access	Manual browser entry to /predict_data (GET method)	Method Not Allowed / Redirect / Error Message	Pass
TC11	Security Test	Attempt to inject script via form	Input sanitization / No script execution	Pass
TC12	Missing Model File	Manually delete model file and submit form	Graceful error: "Model not found"	Pass
TC13	API Response Time	Timed prediction request	Prediction completes within 1–2 seconds	Pass
TC14	Cross-Browser Compatibility	Form accessed via Chrome, Firefox, Edge	UI displays consistently across browsers	Pass
TC15	Mobile Responsiveness	Access form from mobile browser	Layout adjusts and remains usable	Pass

7. CONCLUSION AND FUTURE ENHANCEMENT

7.1 Project Conclusion

This project successfully implements a robust and accessible early screening tool for **Autism Spectrum Disorder (ASD)** in toddlers by harnessing the power of advanced machine learning techniques, specifically the **XGBoost Classifier**. The system is designed to analyze both **behavioral responses**—sourced from the widely recognized Q-Chat-10 questionnaire—and **demographic factors** such as age, sex, ethnicity, family history, and jaundice presence. These inputs are processed and used to predict the likelihood of ASD with notable precision and reliability. The incorporation of a machine learning model enhances the diagnostic sensitivity by uncovering hidden patterns in the data that may not be immediately obvious to caregivers or clinicians.

To ensure usability and broad accessibility, a **web-based application** was developed using the Flask framework. This interface is clean, responsive, and intuitive, allowing users—typically parents, guardians, or healthcare workers—to easily enter relevant information and receive real-time predictive feedback. The results include not only a binary classification (ASD or not) but also a detailed **risk analysis**, highlighting specific questionnaire items that indicate concern, the total risk score, and the overall risk level (e.g., low or high). Such feedback fosters better understanding and prompts users to seek timely clinical evaluation if necessary.

Importantly, this project addresses a significant gap between **early caregiver concerns** and the typically delayed timeline of professional ASD diagnosis. Many families often wait months—or even years—for access to formal assessments. By offering a **quick, cost-effective, and scalable solution**, this tool democratizes early screening, particularly in underserved or resource-limited communities. Early identification of developmental disorders is known to have a profound impact on long-term outcomes, enabling more effective early interventions that can enhance social, cognitive, and emotional development.

Furthermore, the architecture of the system is **modular and extensible**, meaning it can be updated with more advanced models or new datasets as additional research becomes available. It can also be integrated into larger digital health ecosystems, electronic health records, or school screening systems with minimal adaptation. This opens the door for **future improvements**, such as adding speech or facial behavior analysis, multilingual support, or integration with voice assistants and mobile platforms.

In conclusion, the Autism Detection System developed in this project is not just a technical achievement, but a step forward in the application of **AI for social good**. It empowers caregivers by enhancing awareness of developmental delays, fosters proactive health-seeking behavior, and contributes to reducing the time to diagnosis. By bridging the gap between observation and action, this tool has the potential to improve outcomes for children with autism and support their families in meaningful ways.

7.2 Future Enhancements

To expand the system's capabilities and increase its impact, several enhancements are proposed:

- **Incorporation of Multimodal Behavioral Data:**

Future versions could include analysis of additional behavioral signals such as speech patterns, facial expressions, eye gaze tracking, and gesture recognition. Integrating these features using computer vision and audio processing techniques would provide a more comprehensive assessment of the toddler's social and communicative behavior.

- **Wider Age Range Screening:**

Currently focused on toddlers, the tool can be extended to cover infants and adolescents, addressing the full developmental spectrum and detecting ASD signs at different life stages. This expansion would require retraining models with age-appropriate data and questions.

- **Interactive Chatbot for Guidance and Support:**

Implementing an AI-powered chatbot would enhance user experience by providing real-time assistance, answering caregiver questions, clarifying questionnaire items, and offering personalized recommendations. This would make the tool more engaging and easier to navigate, especially for first-time users.

- **Cloud-Based Deployment for Scalability:**

Hosting the system on cloud infrastructure would enable access from multiple locations and devices, support a larger number of concurrent users, and facilitate centralized data collection for continuous model improvement and large-scale epidemiological studies.

- **Collaboration with Healthcare Professionals:**

Establishing partnerships with pediatricians, psychologists, and ASD specialists would enable clinical validation of the tool, ensuring its accuracy and reliability in real-world settings. Such collaborations could also lead to the development of tailored intervention pathways integrated into the system.

- **Multilingual and Accessibility Enhancements:**

Adding support for multiple languages and voice input/output will broaden accessibility to non-English-speaking populations and users with visual or motor impairments. These features promote inclusivity and ensure the tool can serve diverse communities effectively.

- **Data Privacy and Security Improvements:**

As the system collects sensitive health-related information, future enhancements will focus on implementing strong data encryption, secure authentication, and compliance with healthcare data regulations (such as HIPAA or GDPR) to protect user privacy and build trust.

- **Integration with Health Records and Referral Systems:**

Future work could enable seamless integration with electronic health records (EHR) and referral systems, allowing healthcare providers to track patient progress, schedule evaluations, and coordinate early interventions based on screening results.

Through these planned enhancements, the project aims to evolve from a standalone screening tool into a comprehensive, accessible, and clinically validated platform that supports early detection and intervention for ASD across populations.

8. REFERENCES

8.1 Journals

- [1] Chawla, D. & Sharma, A. "Machine Learning Approaches to Autism Detection," *International Journal of Computer Applications*, 2021.
- [2] Thabtah, F., "Autism Spectrum Disorder Detection: Machine Learning Models and Feature Selection," *Health Informatics Journal*, 2019.

8.2 Books

- [1] Matson, J.L., *Autism Spectrum Disorders: Identification, Education, and Treatment*, Springer, 2017.
- [2] Amaral, D.G., *The Neuroscience of Autism Spectrum Disorders*, Academic Press, 2010.

8.3 Sites

- [1] [CDC Autism Information](#)
- [2] [Autism Speaks](#)
- [3] [UCI Machine Learning Repository - ASD Dataset](#)